Cipher Maps: Total Functions as Trapdoor Approximations

Alexander Towell
lex@metafunctor.com

March 2026

Abstract

Privacy in encrypted computation need not come from access-pattern hiding (ORAM), exact algebraic homomorphism (FHE), or simulation-based security (garbled circuits). We show that a *cipher map*—a total function on bit strings where the concepts of domain, correctness, and function identity exist only behind a one-way trapdoor—provides a distinct, quantifiable privacy model with predictable composition and information-theoretically optimal space. We characterize cipher maps by four independent, measurable properties (totality, representation uniformity, correctness, composability), prove the composition theorem governing error accumulation, and present three constructions—HashSet, entropy map, and trapdoor Boolean algebra—spanning batch and online strategies. The Singular Hash Map achieves the information-theoretic lower bound of $-\log_2 \varepsilon + H(Y)$ bits per element.

1 Introduction

Consider a setting in which a trusted party wishes to outsource computation to an untrusted machine. The trusted party holds a function $f : X \to Y$—a lookup table, a set membership predicate, an index—and wants the untrusted machine to evaluate f on encoded inputs without learning f, X, or Y.

The central object of this paper is the *cipher map*: a total function $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ that the untrusted machine evaluates blindly. The key insight is that $\hat{f}$ is total—every n-bit input produces an n-bit output, with no concept of "in-domain" or "out-of-domain." The notions of domain, correctness, false positive, and false negative exist only on the trusted machine, which holds the encoding and decoding functions (the trapdoor). The untrusted machine sees only opaque bit strings flowing through opaque total functions.

This paper develops the theory of cipher maps in a self-contained way. We define four properties that characterize a cipher map (§4), instantiate them against three concrete constructions from distinct domains (§6), formalize the trusted/untrusted machine model (§5), present the Singular Hash Map achieving space-optimal performance (§7), and prove the composition theorem governing error accumulation (§8).

Bernoulli error model. The error framework underlying cipher maps is the Bernoulli model [15]. Two axioms (element-wise independence and conditional independence of block error rates) reduce the error model from exponential complexity to two parameters per element. This tractability is what makes the composition theorem (Property 4) possible: errors combine predictably because they are independent across elements. The Bernoulli model is developed independently; we reference it here only for the error accounting that cipher maps inherit.

What this is not. Cipher maps are not oblivious RAM [12]: ORAM reshuffles storage to hide access patterns with polylogarithmic overhead, while cipher maps use static total functions with noise injection. Cipher maps are not fully homomorphic encryption [11]: FHE preserves exact algebraic structure on ciphertexts, while cipher maps are approximate and hash-based. The closest relative is garbled circuits [16]—both use encrypted lookup tables—but garbled circuits are one-time use and exact, while cipher maps are reusable and approximate with noise closure.

2 Related Work

Cipher maps draw on and differ from several lines of work in data structures, cryptography, and secure computation.

Membership data structures. Bloom filters [5] are the canonical approximate membership structure: a bit array with k hash functions, trading false positives for space. Quotient filters [4] and cuckoo filters [9] improve cache locality and support deletion, respectively, but share the same basic model. The HashSet construction (§6.1) subsumes Bloom filters as a special case: a single hash function ($k = 1$), exact correctness ($\eta = 0$), and the same information-theoretic space bound of $-\log_2 \varepsilon$ bits per element. The difference is that the HashSet is a cipher map—a total function on bit strings with a trapdoor—while Bloom filters expose their output directly.

Perfect hash functions. Fredman, Komlós, and Szemerédi [10] introduced FKS perfect hashing with $O(n)$ space and $O(1)$ lookup. Belazzougui, Botelho, and Dietzfelbinger [2] achieved minimal perfect hash functions near the information-theoretic lower bound. The Singular Hash Map (§7) is a perfect hash function with a different optimization target: it maps domain elements to prefix-free codewords (not to consecutive integers) and trades correctness ($\eta > 0$) for space, achieving $(1 - \eta)(-\log_2 \varepsilon + H(Y))$ bits per element.

Property-preserving encryption. Order-preserving encryption (OPE) [1, 6] and deterministic encryption [3] enable computation on ciphertexts by preserving algebraic structure (order, equality). Cipher maps differ fundamentally: rather than preserving structure at the cost of leakage, cipher maps hide structure behind a total function. The untrusted machine cannot distinguish domain elements from noise, and the output distribution is controlled by the representation uniformity parameter δ.

Searchable symmetric encryption. Song, Wagner, and Perrig [14] introduced practical encrypted keyword search. Curtmola et al. [8] formalized SSE with simulation-based security (IND-CKA). Cash et al. [7] extended SSE to Boolean queries with sublinear search. SSE constructions use inverted indices and game-based security definitions; cipher maps use total functions with information-theoretic parameters (η, ε, δ). The approaches address different threat models: SSE hides query results from the server (up to leakage profiles), while cipher maps hide function identity and domain membership through totality and noise closure.

Garbled circuits and fully homomorphic encryption. Yao's garbled circuits [16] evaluate arbitrary functions on encrypted inputs via encrypted lookup tables. Gentry's fully homomorphic encryption [11] enables exact computation on ciphertexts without interaction. Both achieve exact computation on encrypted data. Cipher maps trade exactness for simplicity: a cipher map is a static lookup table (one hash evaluation per query) with tunable error η, whereas garbled circuits

require per-gate encryption and are single-use, and FHE incurs substantial computational overhead from noise management in lattice operations.

Homophonic substitution. The representation uniformity property (multiple encodings per value, $K(x) \propto 1/D(x)$) is a modern instance of homophonic substitution [13], in which frequent plaintext symbols are assigned multiple ciphertext equivalents to flatten frequency distributions. The connection is direct: representation uniformity generalizes homophonic substitution from substitution ciphers to arbitrary total functions.

3 The Cipher Map Abstraction

3.1 Preliminaries

Throughout, $h : \{0,1\}^* \to \{0,1\}^n$ denotes a cryptographic hash function modeled as a *random oracle* [**?**]: a truly random function accessible to all parties via oracle queries. Results stated "under the random oracle model" assume h has this property. Elements of any finite type are encoded as bit strings before hashing.

3.2 Definition

Definition 3.1 (Cipher map)**.** Let $f : X \to Y$ be a *latent function.* A *cipher map* for f is a tuple $(\hat{f}, \mathrm{enc}, \mathrm{dec}, s)$ where:

1. $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is a **total function** on n-bit strings;
2. $\mathrm{enc} : X \times \{0, \ldots, K(x)-1\} \to \{0,1\}^n$ is an encoding function, where $K(x) \geq 1$ is the number of encodings for element x;
3. $\mathrm{dec} : \{0,1\}^n \to Y \cup \{\perp\}$ is a decoding function;
4. s is a secret (the trapdoor) from which $\hat{f}$, enc, and dec are derived.

The untrusted machine holds $\hat{f}$. The trusted machine holds enc, dec, and s.

A cipher value $\mathrm{enc}(v, k)$ can be viewed as a cipher map for the constant function $f_v(x) = v$: under this view, cipher values and cipher maps are the same kind of object—a total function on bit strings—simplifying the untrusted machine's interface.

3.3 Two Construction Strategies

Cipher maps arise from two fundamentally different construction strategies:

Batch construction. The trusted machine invests computation upfront to find a seed s such that $\hat{f}$ satisfies correctness constraints for all $x \in X$ simultaneously. This gives tunable correctness parameter η and optimal space, but requires knowing X and f at construction time. Examples: HashSet (§6.1), entropy map (§6.2).

Online construction. The cipher map $\hat{f}$ is defined directly by the hash structure—no seed search is needed. The secret s is a hash key, not a searched seed. Operations are limited to those the hash structure supports algebraically (e.g., set union, intersection). Space and accuracy trade-offs are fixed by the hash width n. Example: trapdoor Boolean algebra (§6.3).

Both strategies produce objects satisfying Definition 3.1.

3.4 Construction Layers

The four properties (defined in §4) arise from three orthogonal type transformations applied to the latent function.

Layer 1: Undefined injection. Extend the domain X to $X \cup \{\perp_1, \ldots, \perp_N\}$ by adding N undefined elements. A function $f : X \to Y$ lifts to a partial function $\tilde{f}$ that is undefined on the $\perp_i$. Real elements become a fraction $|X|/(|X| + N)$ of the extended type. This controls the space parameter ε (the probability that random bits form a valid codeword).

Layer 2: Noise closure. Lift the partial function $\tilde{f}$ to a total function $\hat{f}$ by mapping undefined inputs to random outputs via cryptographic hash. The untrusted machine cannot distinguish real inputs from undefined inputs—both produce n-bit output. This yields **Property 1 (Totality)**.

Layer 3: Multiple representations. Replace each value x with $K(x) \geq 1$ distinct representations, keyed by the secret s. Representational equality implies value equality, but value equality does not imply representational equality. Assigning $K(x) \propto 1/D(x)$ equalizes frequencies. This yields **Property 2 (Representation Uniformity)**.

The combined type is cipher(noise(undef(X)))—an element with multiple representations, total evaluation, and diluted domain. Properties 3 (Correctness) and 4 (Composability) are emergent: correctness depends on the construction method; composability depends on the totality guarantee enabling chained evaluation.

These layers are conceptual scaffolding explaining *why* each property exists, not algebraic monads. Whether the layers satisfy formal monad laws in the approximate setting is an open question (see §10).

4 Four Properties

A cipher map $(\hat{f}, \mathrm{enc}, \mathrm{dec}, s)$ for $f : X \to Y$ is characterized by four properties parameterized by $(\eta, \varepsilon, \mu, \delta)$.

4.1 Property 1: Totality

Definition 4.1 (Totality)**.** The function $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is *total*: for every $c \in \{0,1\}^n$, $\hat{f}(c) \in \{0,1\}^n$. For c chosen uniformly at random from $\{0,1\}^n \setminus \mathrm{Im}(\mathrm{enc})$, the distribution of $\hat{f}(c)$ is uniform over $\{0,1\}^n$ under the random oracle model.

Totality ensures that the untrusted machine cannot distinguish a real query $\mathrm{enc}(x, k)$ from a random filler string $c \leftarrow \{0,1\}^n$, because both produce n-bit outputs. If out-of-domain queries returned an error or $\perp$, the untrusted machine could identify real queries by observing which inputs produce valid output.

4.2 Property 2: Representation Uniformity (δ-bounded)

Definition 4.2 (Representation uniformity)**.** Let D be a distribution on X. Define the induced distribution on cipher values:

$$Q(c) = \sum_{x \in X} D(x) \cdot \frac{|\{k : \mathrm{enc}(x, k) = c\}|}{K(x)}. \tag{1}$$

The cipher map has *δ-representation uniformity* if

$$d_{\mathrm{TV}}(Q, \mathrm{Uniform}(\{0,1\}^n)) \leq \delta, \tag{2}$$

where d_{TV} is total variation distance.

Total variation distance has the operational interpretation that no statistical test can distinguish Q from uniform with advantage exceeding δ. To achieve small δ, assign $K(x) \propto 1/D(x)$ encodings per value (homophonic substitution), so that frequent values get more representations.

Remark 4.1 (Marginal uniformity only). Representation uniformity bounds the *marginal* distribution of individual cipher values. It says nothing about *joint* distributions: an adversary observing sequences of cipher values can still detect correlations. See §9.1 for the encoding granularity tradeoff.

4.3 Property 3: Correctness (η-bounded)

Definition 4.3 (Correctness)**.** For x chosen uniformly from X and k chosen uniformly from $\{0, \ldots, K(x) - 1\}$:

$$\Pr_{x,k}\big[\mathrm{dec}(\hat{f}(\mathrm{enc}(x,k))) \neq f(x)\big] \leq \eta. \tag{3}$$

The parameter $\eta \in [0, 1)$ is a *construction* parameter: it reflects how many constraints the seed s must satisfy. Tolerating $\eta > 0$ makes the seed search faster and the structure smaller. Once built, the failing elements are deterministic for a given seed—the same elements always fail.

Remark 4.2 (Approximation as privacy). Nonzero η also provides privacy: a false positive creates plausible deniability, since the untrusted machine cannot distinguish "element is in the set" from "cipher map erred." With $\eta = 0$, the answer is deterministic; with $\eta > 0$, each answer carries ambiguity.

4.4 Property 4: Composability

Definition 4.4 (Composability)**.** The composition of two cipher maps is again a cipher map. Specifically: let $(\hat{f}, \mathrm{enc}_f, \mathrm{dec}_f, s_f)$ be a cipher map for $f : X \to Y$ and $(\hat{g}, \mathrm{enc}_g, \mathrm{dec}_g, s_g)$ a cipher map for $g : Y \to Z$. Then $\hat{g} \circ \hat{f} : \{0,1\}^n \to \{0,1\}^n$, defined by $(\hat{g} \circ \hat{f})(c) = \hat{g}(\hat{f}(c))$, with encoding enc_f and decoding dec_g, is a cipher map for $g \circ f : X \to Z$.

Composability is enabled by totality: since $\hat{f}$ is total, its output is always a valid n-bit input to $\hat{g}$. The untrusted machine can chain cipher maps without needing to decode intermediate results.

Theorem 4.1 (Composition correctness)**.** *Under the hypotheses of Definition 4.4, if $\hat{f}$ has correctness η_f and $\hat{g}$ has correctness η_g, then $\hat{g} \circ \hat{f}$ has correctness:*

$$\eta_{g \circ f} = 1 - (1 - \eta_f)(1 - \eta_g) = \eta_f + \eta_g - \eta_f \eta_g. \tag{4}$$

Proof. See §8 (Theorem 8.2). □

Remark 4.3 (Noise under composition). Noise input to $\hat{f}$ produces random output (by totality). With probability ε, that random output happens to be a valid codeword for $\hat{g}$. We do not try to control noise output—the cipher map is just a hash. This is a feature: valid-looking outputs do not prove valid inputs.

Table 1: Cipher map parameters.

Parameter	Range	Controls	Determined by
η (correctness)	$[0, 1)$	Fraction of wrong answers	Construction (seed search)
ε (noise decode)	$(0, 1)$	Pr[random bits valid]	Encoding scheme
μ (value cost)	$(0, \infty)$	Bits per element for values	$\mu = H(Y)$
δ (uniformity)	$[0, 1]$	TV distance from uniform	Multiplicity $K(x)$
$K(x)$ (multiplicity)	$\{1, 2, \ldots\}$	Encodings per element	Set to $\propto 1/D(x)$
p (entanglement)	$\{1, \ldots, k\}$	Encoding granularity	Correlation structure (§9.1)

4.5 Parameter Decomposition

The space per element decomposes as:

$$\text{bits/element} = -\log_2 \varepsilon + \mu = -\log_2 \varepsilon + H(Y), \tag{5}$$

where $-\log_2 \varepsilon$ bits distinguish signal from noise and $H(Y)$ bits encode the function value. This decomposition is information-theoretic.

5 The Trusted/Untrusted Machine Model

The four properties take operational meaning through a two-party model.

Definition 5.1 (Trusted machine T)**.** The trusted machine T holds: the seed s (trapdoor), the encoding function enc, the decoding function dec, and knowledge of which queries are real vs. filler.

T can:

1. Encode plaintext values: $x \mapsto \text{enc}(x, k)$ for chosen k.
2. Decode cipher results: $r \mapsto \text{dec}(r)$.
3. Inject noise: generate random $c \leftarrow \{0, 1\}^n$ as filler queries.
4. Verify results: check $\text{dec}(\hat{f}(\text{enc}(x, k))) = f(x)$.
5. Reseed: construct a new cipher map with fresh s' when leakage accumulates.

Definition 5.2 (Untrusted machine U)**.** The untrusted machine U holds: the cipher map $\hat{f}$ (a total function on bit strings) and a collection of cipher values (encodings + filler, indistinguishable to U).

U can:

1. Evaluate cipher maps: $c \mapsto \hat{f}(c)$ for any $c \in \{0, 1\}^n$.
2. Return results to T.

U cannot:

1. Decode cipher values (does not hold s or dec).
2. Distinguish real encodings from filler (by Property 1).
3. Determine the domain X or function f (by Property 2, cipher values are δ-close to uniform).
4. Enumerate which inputs are "in-domain" (by totality, every input produces output).

Protocol. The information flow is:

1. T encodes inputs: $\mathrm{enc}(x_1, k_1), \ldots, \mathrm{enc}(x_m, k_m)$.
2. T generates filler: $c_1, \ldots, c_r \leftarrow \{0,1\}^n$.
3. T sends all cipher values (real + filler, shuffled) to U.
4. U evaluates $\hat{f}$ on each cipher value and returns results.
5. T decodes the results it cares about; discards filler results.

What U observes is a sequence of n-bit strings as input and a sequence of n-bit strings as output. Under the four properties:

- Totality: every input produces output; no error signals leak domain information.
- Representation uniformity: cipher values are δ-close to uniform; frequency analysis is bounded by δ.
- Correctness: irrelevant to U (only T decodes).
- Composability: U can chain cipher maps without decoding intermediate results.

Table 2: What the four properties guarantee against U.

Adversary capability	Prevented by	Mechanism
Distinguish real from filler	Totality	All queries produce output
Frequency analysis	Rep. uniformity	Values δ-close to uniform
Learn function values	Trapdoor	Cannot invert h without s
Enumerate domain X	Totality + trapdoor	Cannot test membership
Detect query correlations	**Not fully**	Only marginal uniformity

The last row is the honest limitation: the four properties do not provide full correlation hiding unless the encoding granularity encompasses the correlated values (§9.1).

6 Concrete Constructions

We instantiate the cipher map abstraction against three constructions. For each, we identify which of the four properties are satisfied and with what parameter values.

6.1 HashSet

The HashSet is the simplest cipher map: an approximate set membership structure derived from a seed search.

Latent function. Set membership indicator: $f = \mathbf{1}_A : X \to \{0,1\}$ where $A \subseteq X$.

Construction. Choose a seed s such that

$$h(x\|s) = 0^n \quad \text{for all } x \in A, \tag{6}$$

where $h : \{0,1\}^* \to \{0,1\}^n$ is a cryptographic hash modeled as a random oracle. Membership test: declare $x \in A$ if and only if $h(x\|s) = 0^n$.

Mapping to cipher map abstraction.

- $\hat{f}(c) = h(c\|s)$, a total function on all bit strings.
- $\mathrm{enc}(x, 0) = x$ (single encoding per element; $K(x) = 1$).
- $\mathrm{dec}(r) = 1$ if $r = 0^n$, else 0.

Parameter instantiation.

Parameter	Value	Justification
η	0	All members hash to 0^n by construction
ε	2^{-n}	Random c satisfies $h(c\|s) = 0^n$ w.p. 2^{-n}
μ	≤ 1	Binary output
δ	N/A	$K(x) = 1$; no representation uniformity
Space	$n = -\log_2 \varepsilon$ bits/element	

Property status. Totality: **yes** (h is defined on all inputs). Representation uniformity: **no** ($K = 1$; cipher values are just domain elements). Correctness: **yes** ($\eta = 0$). Composability: **yes** (intersection of HashSets: $x \in A \cap B$ iff $h(x\|s_A) = 0^n$ and $h(x\|s_B) = 0^n$).

The HashSet demonstrates that the four properties are independent: a construction can satisfy some without others.

Remark 6.1 (Bits per element). The HashSet stores a single seed s, not per-element data. The quantity $\log_2(|U|/m)$ bits per element is the *amortized information content*: the total information required to specify which m elements belong to the set, divided by m. This is standard in the perfect hashing literature.

Construction complexity. Each candidate seed succeeds with probability $2^{-n|A|}$ (all $|A|$ elements must hash to 0^n), so the expected number of trials is $2^{n|A|}$. The space is $n|A|$ bits total, or $n = -\log_2 \varepsilon$ bits per element, matching the information-theoretic lower bound for membership (§7.1). The exponential construction time motivates the two-level construction of §7.

6.2 Entropy Map

The entropy map generalizes the HashSet to arbitrary functions $f : X \to Y$, achieving Shannon-entropy-optimal value encoding.

Latent function. Arbitrary $f : X \to Y$ where Y is finite.

Construction. Assign a prefix-free code $C_y \subseteq \{0,1\}^*$ for each $y \in Y$, where the fraction of hash space covered by C_y is proportional to $\Pr_D[f(X) = y]$. Find a seed s (via two-level hashing) such that

$$h(x\|s) \in C_{f(x)} \quad \text{for all } x \in X. \tag{7}$$

Evaluation: compute $h(c\|s)$ and decode via the prefix-free code.

Two-level hash (practical algorithm).

1. A primary hash maps x to a bucket index $j \in \{0, \ldots, b-1\}$.
2. For each bucket, find a local seed s_j such that $h(x\|s_j)$ decodes to $f(x)$ for all x in bucket j.
3. Store s_j in a seed table $H[j]$.
4. Query: compute bucket j (1 hash), evaluate $h(x\|H[j])$ and decode (1 hash). Total: $O(1)$ hash operations.

Construction time. Let ℓ be the number of items in a bucket. Each candidate seed succeeds independently with probability ε^ℓ (all ℓ items must land in valid codewords). The expected number of trials per bucket is $\varepsilon^{-\ell}$, geometric. Total expected construction time: $O(b \cdot \varepsilon^{-\ell})$.

Mapping to cipher map abstraction.

- $\hat{f}(c) = h(c\|H[\text{bucket}(c)])$, total function.
- $\text{enc}(x, 0) = x$ (simplest version; $K(x) = 1$).
- $\text{dec}(r) = y$ if $r \in C_y$ for some y, else $\perp$.

Parameter instantiation.

Parameter	Value	Justification
η	Construction-dependent	Fraction of bucket failures; near-zero with retry
ε	Code-dependent	$\Pr[\text{random bits} \in \bigcup_y C_y]$
μ	$H(Y)$	Shannon entropy of output distribution
δ	From code assignment	If $\lvert C_y \rvert \propto \Pr[f(X) = y]$, approaches uniform
Space	$-\log_2 \varepsilon + H(Y)$ bits/element	

Property status. Totality: **yes**. Representation uniformity: **achievable** (by assigning multiple codes proportional to output frequency). Correctness: **yes** ($\eta \approx 0$, tunable). Composability: **yes** (output is an n-bit string; serves as input to another cipher map).

Proposition 6.1 (Entropy map space)**.** *The space per element of an entropy map for $f : X \to Y$ with output distribution $(p_y)_{y \in Y}$ equals $-\log_2 \varepsilon + H(Y)$ bits, where $H(Y) = -\sum_y p_y \log_2 p_y$.*

Proof. The term $-\log_2 \varepsilon$ is the per-element cost of distinguishing signal from noise: each stored element requires that its hash prefix match a valid codeword, which occurs with probability ε for random inputs. The term $H(Y)$ is the expected code length under Shannon-optimal prefix-free coding, which equals the entropy of the output distribution by Shannon's source coding theorem [**?**]. These two components are additive because the noise-rejection bits and the value-encoding bits occupy disjoint portions of the hash output. □

Remark 6.2 (Connection to Bloom filters). The entropy map for the set-indicator function $\mathbf{1}_A$ is analogous to a Bloom filter: two codewords ("member" / "non-member") allocated hash space in ratio $\varepsilon : (1-\varepsilon)$. The constructions differ—hash-based lookup vs. bit array with k hash functions—but the space bounds are related.

6.3 Trapdoor Boolean Algebra

The trapdoor Boolean algebra is an online construction: no seed search is needed, and set operations are performed via bitwise logic on hash values.

Latent function. Set operations over subsets of a string universe. Let X^* denote the free semigroup over alphabet X. The latent structure is the Boolean algebra $(\mathcal{P}(X^*), \cap, \cup, \complement, \emptyset, X^*)$.

Construction. Define a map $F : \mathcal{P}(X^*) \to \{0,1\}^n$ by:

$$F(\{x_1, \ldots, x_k\}) = h(x_1) \mid h(x_2) \mid \cdots \mid h(x_k), \tag{8}$$

where $\mid$ denotes bitwise OR and $h : X^* \to \{0,1\}^n$ is a cryptographic hash.

The operation correspondence is:

Lattice operation	Bit operation	Exact?
$A \cup B$	$F(A) \mid F(B)$	Yes
$A \cap B$	$F(A) \,\&\, F(B)$	Approximate
$A^\complement$	$\sim F(A)$	Approximate
$\emptyset$	0^n	Yes
X^*	1^n	Yes (limiting)

Why intersection is approximate. $F(A) \,\&\, F(B)$ retains all bit positions set in both $F(A)$ and $F(B)$. But $F(A)$ has bits set by *all* elements of A (not just $A \cap B$), and likewise for $F(B)$. When elements of $A \setminus B$ happen to set the same bit positions as elements of $B \setminus A$ (cross-element hash collisions), those bits survive the AND and appear as spurious bits. Formally, $F(A \cap B) \subseteq F(A) \,\&\, F(B)$ (as bit vectors), with equality only when there are no cross-element collisions.

Why NOT degrades with set size. $\sim F(A)$ flips all bits, giving the positions not set by any element of A. But $F(A^\complement)$ is the OR of hashes of all elements not in A: for large complements, this converges to 1^n. Since $F(A)$ also approaches 1^n for large $|A|$, the NOT approximation $\sim F(A) \approx 0^n$ is poor when $|A|$ is large. NOT is most accurate when $|A|$ is small relative to n.

Relational predicates.

- Membership: $x \in_B W$ iff $h(x) \,\&\, F(W) = h(x)$.
- Subset: $A \subseteq_B B$ iff $F(A) \,\&\, F(B) = F(A)$.

Proposition 6.2 (Membership false positive rate)**.** *For random $x \notin W$ where $|W| = k$, under the random oracle model:*

$$\varepsilon_\in(k, n) = (1 - 2^{-(k+1)})^n. \tag{9}$$

Proof. Under the random oracle model, hash outputs are independent uniform bits. Each bit position j has $\Pr[h(x)_j = 1] = 1/2$ and $\Pr[F(W)_j = 0] = (1/2)^k = 2^{-k}$ (probability none of the k elements set bit j). Bit j causes a rejection (correctly identifying $x \notin W$) when $h(x)_j = 1$ and $F(W)_j = 0$, which happens with probability $2^{-(k+1)}$. All n bits must independently pass for a false positive: $\varepsilon_\in = (1 - 2^{-(k+1)})^n$. □

Space complexity. Single-level: $n = \log \varepsilon / \log(1 - 2^{-(k+1)}) = O(2^k)$ bits. This is exponential in set size. Two-level (practical): hash to 2^w bins of $(q - w)$ bits each. Total space $2^w(q - w)$ bits. False positive rate: $\varepsilon(k, w, q) = (1 - 2^{-(k/2^w+1)})^{q-w}$.

Mapping to cipher map abstraction. For the membership predicate on a set W:

- $\hat{f}(c) = c \,\&\, F(W)$, total function on all n-bit strings.
- $\mathrm{enc}(x, 0) = h(x)$.
- $\mathrm{dec}(r) = 1$ if $r = h(x)$ (where x is the queried element), else 0.

Parameter instantiation (membership predicate, set size k).

Parameter	Value	Justification
η	Operation-dependent	Union exact; intersection approximate
ε	$(1 - 2^{-(k+1)})^n$	Membership FP rate
μ	n bits (fixed width)	Entire set as single n-bit vector
Space	n bits total (not per element)	Constant-size; FP rate degrades with k

Property status. Totality: **yes** (bitwise operations defined on all n-bit strings). Representation uniformity: **marginal only** (see Remark 4.1 and §9.1). Correctness: **partial** (union exact; intersection/NOT approximate). Composability: **yes** (Boolean operations compose; error propagates via interval arithmetic).

Remark 6.3 (Boolean asymmetry). The Boolean algebra homomorphism F preserves union and intersection at the bit level but does not preserve complement: the complement of a finite set in the infinite free semigroup X^* covers all hash values (by pigeonhole, $F(A^\complement) = 1^n$ for any non-trivial A). This asymmetry is structural, not an artifact of the construction, and constrains all cipher map systems that compose Boolean operations over hash-based set representations.

6.4 Construction Comparison

7 The Singular Hash Map

The Singular Hash Map is a concrete cipher map construction achieving the information-theoretic space lower bound. It combines the entropy map's prefix-free coding with a global seed search.

7.1 Information-Theoretic Lower Bound

Theorem 7.1 (Lower bound)**.** *Any data structure representing an approximate map $\hat{f}$ for $f : X \to Y$ with noise decode probability ε and correctness $\eta = 0$ over n stored elements requires at least*

$$n\big(-\log_2 \varepsilon + H(Y)\big) \text{ bits}, \tag{10}$$

Table 3: Comparison of the three constructions.

	HashSet	Entropy Map	Trapdoor Boolean Alg.
Construction	Batch	Batch	Online
Latent function	$\mathbf{1}_A$	Arbitrary $f : X \to Y$	Set ops on $\mathcal{P}(X^*)$
Totality	Yes	Yes	Yes
Rep. Uniformity	No ($K=1$)	Achievable	Marginal only
Correctness η	0	≈ 0 (tunable)	Union exact; $\cap/\neg$ approx.
ε	2^{-n}	Code-dependent	$(1-2^{-(k+1)})^n$
μ	≤ 1 bit	$H(Y)$	n bits
Space/element	$-\log_2 \varepsilon$	$-\log_2 \varepsilon + H(Y)$	n/k (amortized)

or equivalently $-\log_2 \varepsilon + H(Y)$ *bits per element.*

Proof. The bound decomposes into two independent information requirements.

Membership component. Consider the membership predicate $\mathrm{dec}(\hat{f}(c)) \neq \perp$. For a universe U with $|U| \gg n$, the structure must distinguish n stored elements from $|U| - n$ non-elements, with at most an ε fraction of non-elements decoding to valid output. There are $\binom{|U|}{n}$ possible sets of size n from a universe U. Any representation must distinguish all such sets, requiring at least $\log_2 \binom{|U|}{n}$ bits. For $|U| \gg n$, Stirling's approximation gives $\log_2 \binom{|U|}{n} \approx n \log_2(|U|/n)$ bits, or $\log_2(|U|/n)$ bits per element. Setting $\varepsilon = n/|U|$ (the fraction of the universe that decodes validly), this is $n \log_2(1/\varepsilon)$ bits.

Value component. Independently of membership, the structure must encode the function values $(f(x_1), \ldots, f(x_n))$. By Shannon's source coding theorem [**?**], encoding n independent draws from the distribution (p_y) requires at least $nH(Y)$ bits.

Since membership and value encoding are jointly necessary, the total requirement is $n(-\log_2 \varepsilon + H(Y))$ bits. The additivity holds because the membership bits and value-encoding bits convey independent information: by the entropy chain rule, $H(\text{membership}, \text{value}) = H(\text{membership}) + H(\text{value} \mid \text{membership})$, and conditioning on membership (i.e., knowing which elements are stored) does not reduce the entropy of the value assignment, since the values $(f(x_1), \ldots, f(x_n))$ are an independent degree of freedom. □

7.2 Acceptance Predicates

The construction depends on a rule for deciding whether a hash value encodes a particular output. We abstract this as an *acceptance predicate*.

Definition 7.1 (Acceptance predicate)**.** An acceptance predicate for codomain Y is a family of disjoint sets $\{A(y) \subseteq \{0,1\}^n : y \in Y\}$. The decoder is $\mathrm{dec}(\mathrm{hash}) = y$ if $\mathrm{hash} \in A(y)$, and $\perp$ otherwise. The acceptance probability for value y is $\alpha(y) = |A(y)|/2^n$, and the noise-decode probability is $\varepsilon = \sum_y \alpha(y) = |\bigcup_y A(y)|/2^n$.

Different predicates give different space and construction-time trade-offs:

- **Prefix-free code.** $A(y) = \{c \in \{0,1\}^n : c \text{ starts with codeword}(y)\}$. Acceptance probability $\alpha(y) = 2^{-|\mathrm{code}(y)|}$. Shannon-optimal when $|\mathrm{code}(y)| \approx -\log_2 p_y$. This is the entropy map construction.
- **Threshold.** $A(y) = \{c : c < t(y)\}$ for thresholds $0 = t_0 < t_1 < \cdots$. Acceptance probability $\alpha(y) = (t_{y+1} - t_y)/2^n$. Allows fine-grained control of per-value acceptance rates.

- **Range partition.** Partition $[0, 2^n)$ into contiguous ranges $[l_y, l_y + w_y)$. Setting $w_y \propto p_y$ achieves Shannon-optimal allocation with uniform acceptance probability.

All three achieve the same space bound $-\log_2 \varepsilon + H(Y)$ bits per element when acceptance probabilities are Shannon-optimal ($\alpha(y) \propto p_y$). They differ in construction time and implementation simplicity.

7.3 Construction

Algorithm 1 Singular Hash Map Construction

```
function BuildCipherMap(M, A, h)                    ▷ A is an acceptance predicate
    ℓ ← 0
    while true do
        success ← true
        for (x, y) ∈ M do
            hash ← h(ℓ) ⊕ h(x)                      ▷ XOR combines seed with key
            if hash ∉ A(y) then
                success ← false; break
            end if
        end for
        if success then
            return CipherMap(h, A, ℓ)
        end if
        ℓ ← ℓ + 1
    end while
end function
```

The construction searches for a seed ℓ such that for every $(x, y) \in M$, the hash $h(\ell) \oplus h(x)$ falls in the acceptance set $A(y)$. Query evaluation computes $\hat{f}(c) = h(\ell) \oplus h(c)$ and decodes via the acceptance predicate.

Remark 7.1 (Relaxed correctness, $\eta > 0$). Algorithm 1 implements the $\eta = 0$ case. For $\eta > 0$, accept any seed for which at most $\lfloor \eta \cdot m \rfloor$ elements fail. This dramatically reduces construction time (see §**??**).

7.4 Space Optimality

Theorem 7.2 (Space complexity)**.** *The Singular Hash Map achieves space complexity*

$$(1 - \eta)\big(-\log_2 \varepsilon + \mu\big) \text{ bits per element,} \tag{11}$$

where $\mu = H(Y)$ is the mean encoding length. When $\eta = 0$, this matches the lower bound of Theorem 7.1.

Proof. Step 1: False negative reduction. When false negatives are permitted at rate η, only $(1-\eta)n$ of the n elements must decode correctly, reducing the effective element count.

Step 2: Per-element bit cost. For each stored element x_i, the hash $h(\ell) \oplus h(x_i)$ must fall in the acceptance set $A(y_i)$ (Definition **??**). Under the random oracle model, this occurs with probability $\alpha(y_i)$. Shannon-optimal allocation sets $\alpha(y_i) \propto p_{y_i}$, giving per-element information cost $-\log_2 \alpha(y_i) \approx -\log_2 p_{y_i}$ and expected cost per element $\sum_y p_y(-\log_2 \alpha(y)) = \mu = H(Y)$.

Step 3: False positive control. For non-stored elements $x \notin M$, the hash $h(\ell) \oplus h(x)$ is uniformly distributed under the random oracle model. The probability of falling in *any* acceptance set is $\varepsilon = \sum_y \alpha(y)$, the noise-decode probability. Achieving target rate ε requires $-\log_2 \varepsilon$ bits of hash beyond the value encoding.

Step 4: Combining. Each stored element costs $-\log_2 \varepsilon + \mu$ bits, and $(1-\eta)n$ elements are stored, giving total space $(1-\eta)n(-\log_2 \varepsilon + \mu)$ bits. □

Corollary 7.3 (Asymptotic optimality)**.** *When $\eta = 0$, the Singular Hash Map achieves $-\log_2 \varepsilon + H(Y)$ bits per element, matching the information-theoretic lower bound.*

7.5 Construction Time

Proposition 7.4 (Per-seed success probability)**.** *For m stored elements with acceptance predicate A, the probability of successful construction for a given seed ℓ is:*

$$p = \prod_{i=1}^{m} \alpha(y_i), \tag{12}$$

where $\alpha(y_i) = |A(y_i)|/2^n$ is the acceptance probability for the i-th element's target value. The expected number of trials is $1/p$.

Proof. Under the random oracle model, the hash values $h(\ell) \oplus h(x_i)$ for distinct x_i are independent uniformly distributed bit strings. For element x_i, the hash falls in $A(y_i)$ with probability $\alpha(y_i)$. Independence across elements gives the product formula. The number of trials is geometric with success probability p. □

For uniform acceptance probability $\alpha(y) = \varepsilon/|Y|$, this gives $p = (\varepsilon/|Y|)^m$ and expected trials $(|Y|/\varepsilon)^m$—intractable for even modest m.

Bucketed construction. The intractability of the global seed search is not inherent. Partition the m elements into k buckets (e.g., by $h(x) \bmod k$) and find a separate seed ℓ_j for each bucket.

Proposition 7.5 (Bucketed construction time)**.** *With k buckets of expected size m/k, the expected total construction time is*

$$T(k) = k \cdot \left(\frac{1}{\bar{\alpha}}\right)^{m/k}, \tag{13}$$

where $\bar{\alpha} = \left(\prod_i \alpha(y_i)\right)^{1/m}$ is the geometric mean acceptance probability.

Proof. Each bucket has m/k elements (in expectation). The per-bucket success probability is $\bar{\alpha}^{m/k}$ by Proposition 7.4. Expected trials per bucket: $(1/\bar{\alpha})^{m/k}$. Summing over k independent buckets gives the result. □

The trade-off between construction time and space is controlled by k:

Buckets k	Seeds stored	Construction time	Space overhead
1 (global)	1	$(1/\bar{\alpha})^m$	$O(\log m)/m$ bits/elem
$\sqrt{m}$	$\sqrt{m}$	$\sqrt{m} \cdot (1/\bar{\alpha})^{\sqrt{m}}$	$O(\log m/\sqrt{m})$ bits/elem
m (per-element)	m	$m/\bar{\alpha}$ (linear)	$O(\log(1/\bar{\alpha}))$ bits/elem

At $k = m$, each bucket has one element, construction is linear in m, and the overhead is $\log_2(1/\bar{\alpha})$ bits per element for the per-element seed—the same order as the hash width. This is the regime of practical perfect hash functions [2].

8 Composition

8.1 Warm-Up: The AND Gate

Before proving the general composition theorem, we analyze a concrete case: the AND gate operating on two independent Bernoulli Boolean values B_1, B_2 with correctness probabilities $p_1 = 1 - \eta_1$ and $p_2 = 1 - \eta_2$.

Proposition 8.1 (AND gate correctness)**.** *The correctness probability of* $\mathrm{AND}(B_1, B_2)$ *as an approximation of* $\mathrm{AND}(x_1, x_2)$ *depends on the latent inputs:*

x_1	x_2	$\mathrm{AND}(x_1, x_2)$	$\Pr[\textit{correct}]$
1	1	1	$p_1 p_2$
1	0	0	$1 - p_1(1 - p_2)$
0	1	0	$1 - p_2(1 - p_1)$
0	0	0	$p_1 + p_2 - p_1 p_2$

Proof. We verify the $(1, 0)$ case; the others are analogous. When $x_1 = 1$, $x_2 = 0$, the correct output is 0. The output is wrong ($= 1$) only when $B_1 = 1$ (correct, probability p_1) *and* $B_2 = 1$ (incorrect, probability $1 - p_2$). So $\Pr[\text{wrong}] = p_1(1 - p_2)$ and $\Pr[\text{correct}] = 1 - p_1(1 - p_2) = 1 - p_1 + p_1 p_2$. □

The output is a *4th-order Bernoulli Boolean*: four distinct correctness probabilities despite the Boolean type having only two values. Rather than tracking all four, we can bound them by an interval $[\min_{\text{cases}} p_{\text{correct}},\ \max_{\text{cases}} p_{\text{correct}}]$. For AND with $p_1, p_2 \in (0.5, 1)$: minimum $p_1 p_2$ (case $(1, 1)$); maximum $p_1 + p_2 - p_1 p_2$ (case $(0, 0)$).

8.2 General Composition Theorem

Theorem 8.2 (Composition correctness)**.** *Let* $\hat{f}$ *be a cipher map for* $f : X \to Y$ *with correctness* η_f*, and* $\hat{g}$ *a cipher map for* $g : Y \to Z$ *with correctness* η_g*. Then* $\hat{g} \circ \hat{f}$ *is a cipher map for* $g \circ f$ *with correctness*

$$\eta_{g \circ f} \leq \eta_f + \eta_g - \eta_f \eta_g = 1 - (1 - \eta_f)(1 - \eta_g). \tag{14}$$

Equality holds when $\hat{f}$ *and* $\hat{g}$ *use independent seeds and* $\hat{g}$*'s correctness on* $\hat{f}(c)$ *is independent of whether* $\hat{f}$ *was correct (the re-randomization condition).*

Proof. The composition is correct when *both* maps produce correct results.

Upper bound. By the union bound, $\Pr[\text{incorrect}] \leq \eta_f + \eta_g$. Since $\eta_f \eta_g \geq 0$, the tighter bound $\eta_f + \eta_g - \eta_f \eta_g$ also holds.

Equality under re-randomization. If the re-randomization condition holds, $\Pr[\text{both correct}] = (1 - \eta_f)(1 - \eta_g)$, giving $\eta_{g \circ f} = 1 - (1 - \eta_f)(1 - \eta_g)$.

In practice, $\hat{f}$ produces a *specific* encoding of $f(x)$, not a uniformly random one. If $\hat{g}$'s correctness depends on which encoding it receives, the errors may be positively correlated, making the inequality strict. For specific circuits, the case-by-case interval arithmetic from §8.1 gives tighter bounds. □

Remark 8.1 (Garbage propagation). When $\hat{f}$ produces an incorrect output on some input, the value fed to $\hat{g}$ is effectively random. Since $\hat{g}$ is a total function, it produces *some* output on this random input, which is correct with probability at most ε_g (the noise-decode probability of $\hat{g}$). Thus the bound $\eta_{\text{total}} \leq \eta_f + \eta_g - \eta_f \eta_g$ is an upper bound; the actual error rate may be marginally lower due to accidental correctness on garbage inputs. For small η_f, η_g, this distinction is negligible.

8.3 Composition Chains

Corollary 8.3 (Chain composition)**.** *For a chain of m cipher maps $\hat{f}_1, \ldots, \hat{f}_m$:*

$$\eta_{\text{total}} = 1 - \prod_{i=1}^{m}(1 - \eta_i). \tag{15}$$

If all $\eta_i = \eta$:

$$\eta_{\text{total}} = 1 - (1 - \eta)^m \approx m\eta \quad \textit{for small } \eta. \tag{16}$$

Proof. By induction on Theorem 8.2. The base case $m = 2$ is the theorem. For the inductive step, the composition of $m - 1$ maps has correctness $1 - \prod_{i=1}^{m-1}(1 - \eta_i)$; composing with $\hat{f}_m$ gives $1 - \prod_{i=1}^{m-1}(1 - \eta_i) \cdot (1 - \eta_m) = 1 - \prod_{i=1}^{m}(1 - \eta_i)$. The approximation follows from $\ln(1 - \eta) \approx -\eta$ for small η, giving $(1 - \eta)^m \approx e^{-m\eta} \approx 1 - m\eta$. □

Example 8.1. A circuit of 100 cipher maps each with $\eta = 10^{-6}$: $\eta_{\text{total}} = 1 - (1 - 10^{-6})^{100} \approx 10^{-4}$.

8.4 Error Accumulation by Gate Type

For specific circuits, the AND gate analysis (Proposition 8.1) can be extended to OR, NOT, and arbitrary Boolean gates. Each gate type induces a different case table. Rather than tracking exact correctness per case, interval arithmetic propagates bounds: at each gate, compute $[\eta_{\min}, \eta_{\max}]$ from the input intervals. This gives input-dependent bounds tighter than the worst-case Theorem 8.2.

9 Representation Uniformity and Encoding Granularity

We measure representation uniformity using total variation distance d_{TV}, which bounds the distinguishing advantage of any statistical test. We use TV rather than max-divergence (D_∞) because the constructions are designed around frequency equalization, which naturally yields TV bounds.

9.1 The Encoding Granularity Principle

Representation uniformity is defined *per cipher map*: the *encoding granularity* is the domain type X, and uniformity applies to the distribution over encodings of individual elements of X. What correlations are hidden depends on what is encoded as a unit. Consider encrypting a pair of Boolean values (a, b) where a and b are correlated.

Component-wise encoding ($p = 1$). Encode a and b as separate cipher values. Each component's marginal distribution is δ-close to uniform, but joint correlations leak.

Pair-wise encoding ($p = 2$). Encode (a, b) as a single cipher value. The distribution over pair encodings is δ-close to uniform; correlations between a and b are hidden. Cost: domain is $\{0, 1\}^2$ (4 elements) rather than $\{0, 1\}$ (2 elements); space per encoding grows.

Whole-state encoding ($p = k$). Encode the entire program state as one cipher value. All correlations are hidden. Cost: domain is the full state space, typically prohibitive.

Definition 9.1 (Entanglement parameter)**.** For a system encoding k correlated Boolean values, the *entanglement parameter* p is the number of values encoded as a single unit. The system has type $\text{cipher}(\{0, 1\}^p)^{k/p}$, where each block of p bits is encoded jointly.

Proposition 9.1 (Granularity and privacy)**.** *Let $\hat{f}$ be a cipher map for $f : X_1 \times X_2 \to Y$ with representation uniformity δ. Then the marginal distribution of* $\mathrm{enc}((x_1, x_2), k)$ *over random $(x_1, x_2) \sim D$ and random k is δ-close to uniform, and the joint distribution of (x_1, x_2) is hidden up to δ.*

Conversely, let $\hat{f}_1, \hat{f}_2$ be independent cipher maps for $f_1 : X_1 \to Y_1$ and $f_2 : X_2 \to Y_2$, each with representation uniformity δ. Then each marginal cipher value is δ-close to uniform, but correlations between x_1 and x_2 are ***not hidden****, even when $\delta = 0$.*

Proof. The first part follows directly from Definition 4.2 applied to the joint cipher map. For the second part: even with $\delta = 0$ (perfect marginal uniformity), enc_i is a deterministic function of (x_i, k_i). An adversary observing N draws of $(\mathrm{enc}_1(x_1, k_1), \mathrm{enc}_2(x_2, k_2))$ can estimate the joint distribution and recover the correlation structure of (x_1, x_2), because the correlation structure under D is preserved in the joint distribution of cipher value pairs. □

The trade-off. Coarser encoding granularity (larger p) yields better privacy but requires more space and construction time. Composition exists precisely because encoding the entire program as a single cipher value is impractical: we compose smaller cipher maps and accept that intermediate correlations may leak.

Honest limitations.

1. Marginal uniformity says nothing about joint distributions.
2. Encoding granularity is a design choice, not a property of the formalism.
3. Equality pattern leakage is fundamental: any deterministic encoding leaks the equality pattern of its inputs. Multiple representations ($K(x) > 1$) reduce but do not eliminate this.

10 Discussion

10.1 Relationship to the Bernoulli Model

The correctness parameter η of a cipher map corresponds directly to the error rate in a Bernoulli approximation. A cipher map with correctness η induces a Bernoulli approximation of the latent function f: for each $x \in X$, $\Pr[\mathrm{dec}(\hat{f}(\mathrm{enc}(x, k))) \neq f(x)] = \eta(x)$, where the failing elements are deterministic for a given seed.

The composition formula $\eta_{g \circ f} = 1 - (1 - \eta_f)(1 - \eta_g)$ is the standard error accumulation for independent Bernoulli events, identical to the formula derived in the noisy gates framework for AND gates over Bernoulli Booleans. This is not coincidental: a cipher map *is* a Bernoulli approximation, viewed through the lens of hash-based total functions.

The Bernoulli framework organizes approximation into a layered type hierarchy: Bool $\to$ Set $\to$ Map $\to$ Relation $\to$ Type, where each level inherits the error model from the level below. The formalism presented here covers maps ($X \to Y$); relations and algebraic types (sum types, product types) require the extensions developed in the Bernoulli relations and algebraic types work [15]. The BHF (Bernoulli Hash Function) construction from that framework is the optimal cipher map for the batch strategy: it achieves the $-\log_2 \varepsilon + H(Y)$ lower bound with $O(1)$ query time.

10.2 Algebraic Structure

The cipher map abstraction relates to algebraic concepts from the theory of monoid homomorphisms. The decoding function dec is (by construction) a left inverse of enc, and for cipher maps that preserve algebraic structure (e.g., the trapdoor Boolean algebra), dec is a homomorphism from the cipher monoid to the latent monoid—but only up to the equivalence $c_1 \sim c_2 \iff \mathrm{dec}(c_1) = \mathrm{dec}(c_2)$. The quotient $\{0,1\}^n/\!\sim$ is isomorphic to the latent structure.

We note this connection but do not develop it further. The category-theoretic framing (cipher functors lifting monoids to cipher monoids) is a natural extension but requires careful treatment of the approximate setting: the functor laws hold exactly on the quotient but only approximately on representatives.

10.3 Application: Encrypted Search

Encrypted search—information retrieval over confidential data by an untrusted provider—is a direct application of the cipher map abstraction. The domain-specific vocabulary maps to the general framework as follows.

Search agent = trusted machine T. Holds the trapdoor s, encodes queries, decodes results.

Encrypted search provider = untrusted machine U. Holds cipher maps and cipher values; evaluates $\hat{f}(c)$ for any $c \in \{0,1\}^n$ without decoding.

Information need = latent function $f : X \to Y$. For keyword search, $f = \mathbf{1}_A$ (set indicator); for ranked retrieval, f is a scoring function.

Hidden query = cipher value $\mathrm{enc}(x, k)$. The provider evaluates the cipher map on this value without learning x.

Secure index = cipher map $\hat{f}$ stored on the provider.

The four properties specialize to encrypted search:

1. **Totality**: The provider can evaluate the index on any bit string; filler queries produce output indistinguishable from real queries.
2. **Representation uniformity**: Hidden queries are δ-close to uniform; frequency analysis of query traffic is bounded by δ.
3. **Correctness**: The search agent recovers the correct answer with probability $\geq 1-\eta$; nonzero η provides plausible deniability.
4. **Composability**: Boolean combinations of queries (AND, OR, NOT) compose as cipher map chains with predictable error accumulation.

The correctness parameter η has a precise deniability interpretation for Boolean-valued cipher maps.

Proposition 10.1 (Bayesian deniability)**.** *Let $\hat{f}$ be a cipher map for $f : X \to \{0,1\}$ with correctness η, and let $\pi = \Pr[f(x) = 1]$ be the prior. If the adversary observes the decoded output $y = 1$, then*

$$\Pr[f(x) = 1 \mid y = 1] = \frac{\pi(1-\eta)}{\pi(1-\eta) + (1-\pi)\eta}. \tag{17}$$

For $\eta = 0$, the posterior equals 1 (no deniability). For $\eta = 1/2$, the posterior equals π (observation is useless).

Proof. By Bayes' rule: $\Pr[y = 1 \mid f(x) = 1] = 1 - \eta$ (correct positive) and $\Pr[y = 1 \mid f(x) = 0] = \eta$ (false positive). Applying Bayes' theorem with prior π gives the stated formula. □

10.4 Open Questions

1. **Max-divergence vs. TV distance for δ.** TV distance bounds distinguishing advantage; max-divergence (D_∞) would give per-element bounds. Which is the right metric depends on the threat model.

2. **Tight bounds for intersection and NOT.** The intersection false positive rate from cross-element collisions in the trapdoor Boolean algebra needs a closed-form expression as a function of $|A|$, $|B|$, $|A \cap B|$, and n. For NOT, deriving $\eta_{\mathrm{NOT}}(|A|, n)$ would complete the parameter instantiation.

3. **Adaptive encoding granularity.** Can the entanglement parameter p be made adaptive—encoding pairs only for correlated values while encoding independent values separately?

4. **Noise-to-signal probability under composition.** Through a chain of m cipher maps, what is the probability that a filler query produces a decodable output at the end? Naively $1 - (1 - \varepsilon)^m$, but this assumes independence of the ε events across maps.

5. **Layered construction laws.** Do the three construction layers (undef, noise, cipher) satisfy formal algebraic laws in the approximate setting? If so, this would enable equational reasoning about cipher map constructions.

6. **Sum-type confidentiality.** Encoding $\mathrm{OT}(X + Y)$ (the sum type as a single opaque value) hides which branch was taken but breaks composability, since downstream cipher maps cannot dispatch on the branch without decoding. Encoding $\mathrm{OT}(X) + \mathrm{OT}(Y)$ (separate opaque values with a visible tag) preserves composability but leaks the branch tag to the untrusted machine. Whether this trade-off is inherent requires formal proof.

10.5 What This Framework Is Not

To set clear expectations:

- *Not ORAM.* ORAM hides access patterns by reshuffling storage with polylogarithmic overhead. Cipher maps use static total functions.

- *Not differential privacy.* Differential privacy bounds what can be learned about a *dataset* from a *query answer*. Cipher maps bound what an untrusted evaluator learns about a *function* from *opaque evaluations*.

- *Not simulation-based security.* We do not claim that the untrusted machine's view can be simulated. The guarantees are parameterized (δ, ε) rather than negligible in a security parameter.

Acknowledgments

The constructions in this paper draw on the author's earlier work on the Bernoulli data type library and the trapdoor Boolean algebra.

References

[1] Rakesh Agrawal, Jerry Kiernan, Ramakrishnan Srikant, and Yirong Xu. Order preserving encryption for numeric data. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*, pages 563–574, 2004.

[2] Djamal Belazzougui, Fabiano C Botelho, and Martin Dietzfelbinger. Hash, displace, and compress. In *European Symposium on Algorithms*, pages 682–693. Springer, 2009.

[3] Mihir Bellare, Alexandra Boldyreva, and Adam O'Neill. Deterministic and efficiently searchable encryption. In *Advances in Cryptology–CRYPTO 2007*, pages 535–552. Springer, 2007.

[4] Michael A Bender, Martin Farach-Colton, Rob Johnson, Russell Kraner, Bradley C Kuszmaul, Dzejla Medjedovic, Pablo Montes, Pradeep Shetty, Richard P Spillane, and Erez Zadok. Don't thrash: How to cache your hash on flash. *Proceedings of the VLDB Endowment*, 5(11):1627–1637, 2012.

[5] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[6] Alexandra Boldyreva, Nathan Chenette, Younho Lee, and Adam O'Neill. Order-preserving symmetric encryption. In *Advances in Cryptology–EUROCRYPT 2009*, pages 224–241. Springer, 2009.

[7] David Cash, Stanislaw Jarecki, Charanjit Jutla, Hugo Krawczyk, Marcel-Cătălin Roşu, and Michael Steiner. Highly-scalable searchable symmetric encryption with support for boolean queries. In *Advances in Cryptology–CRYPTO 2013*, pages 353–373, 2013.

[8] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, pages 79–88, 2006.

[9] Bin Fan, Dave G Andersen, Michael Kaminsky, and Michael D Mitzenmacher. Cuckoo filter: Practically better than Bloom. In *Proceedings of the 10th ACM International Conference on emerging Networking Experiments and Technologies*, pages 75–88, 2014.

[10] Michael L Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.

[11] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 169–178, 2009.

[12] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.

[13] Gustavus J Simmons. Symmetric and asymmetric encryption. *ACM Computing Surveys*, 11 (4):305–330, 1979.

[14] Dawn Xiaodong Song, David Wagner, and Adrian Perrig. Practical techniques for searches on encrypted data. In *Proceedings of the 2000 IEEE Symposium on Security and Privacy*, pages 44–55, 2000.

[15] Alexander Towell. Bernoulli types: A framework for approximate computation. Working paper, 2024.

[16] Andrew C Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science*, pages 160–164, 1982.